

Project Support Session

This file is meant for personal use by ali@zarreh.ai only.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Agenda

- Project Problem Statement and Dataset Overview
- Open Q&A
- Project Evaluation Rubric
- Project Submission Guidelines
- FAQs

Project Problem Statement

Business Context

In last-mile logistics, roughly 10% of all shipments encounter delivery exceptions - failed attempts, address mismatches, damaged packages, refused deliveries, and weather delays. Each failed delivery costs in direct reattempt expenses, while the downstream impact is far steeper: consumers stop shopping with a retailer after just 2-3 failed deliveries, and supply chain disruptions cost companies millions per year.

Despite these costs, most organizations still handle exception triage and resolution manually. Operations staff read through status logs and messy driver notes, cross-reference customer profiles and internal playbooks, decide on a resolution action, and draft customer notifications - all under time pressure. Teams rely on support tickets, tribal knowledge, and rigid rule engines that cannot handle the nuanced, multi-factor judgment calls that real exceptions demand. A VIP customer with a perishable package and a broken gate code on a second attempt requires a fundamentally different resolution than a standard customer's first failed attempt on a non-perishable item - but manual processes treat them with the same slow, inconsistent workflow.

Project Problem Statement

Business Context

Key KPIs affected include:

- Exception resolution time (constrained by manual triage throughput)
- Escalation accuracy (missed or unnecessary supervisor escalations)
- Customer communication quality and personalization
- Cost per exception (reattempt costs, spoilage write-offs, unnecessary truck rolls)
- Customer retention and lifetime value

If unaddressed, logistics providers face rising exception-handling costs, inconsistent service quality, preventable customer churn, and loss of competitive ground to players who are already investing in AI-powered exception detection and response automation.

By applying an AI-powered multi-agent system that ingests delivery logs, cross-references customer profiles and locker availability, retrieves resolution policy from an operational playbook, and generates validated decisions with personalized customer communications, logistics providers can automate the full exception-handling pipeline from detection through resolution, with built-in quality validation and supervisor escalation where policy requires it.

Project Problem Statement

Objective

The objective is to build a POC of an AI-powered multi-agent delivery exception handling system for the last-mile delivery operations of a mid-sized retailer that:

- uses the operator's available data to process exceptions end-to-end,
- ingests raw delivery status logs, including noisy, duplicated, and multi-row shipment events, and correctly identifies actionable exceptions versus routine operational noise,
- decides the appropriate resolution action (reschedule, reroute to locker, replace, or return to sender) by reasoning over playbook rules, customer context, package constraints, and locker eligibility, while respecting operational policies around perishable handling, fragile thresholds, and locker capacity.
- escalates to a human supervisor when policy requires it and avoids unnecessary escalations that waste supervisor capacity,
- generates personalized customer notifications with tone, channel, and content calibrated to customer tier and exception severity, and
- produces auditable decisions with step-by-step rationale, critical validation traces, and system evaluation metrics across five dimensions: task completion rate, escalation accuracy, tool call accuracy, reasoning trajectory coherence, and end-to-end latency.

Project Problem Statement

Objective

The end goal is to demonstrate measurable accuracy and consistency on curated exception scenarios sufficient to justify scaling the approach to real-time operations with live delivery feeds, broader exception taxonomies, and multi-region deployment.

Dataset Overview

- Structured backend datasets modeling customer profiles and delivery infrastructure to support personalization, routing, and operational decision-making.

Database table Name	Database table Content
Customers	Customer profile data capturing tier, preferences, and engagement attributes for personalization and decision support.
lockers	Locker infrastructure data detailing location, capacity, and operational constraints for delivery routing decisions.

Dataset Overview

- Structured data files supporting analysis, validation, and decision-making.

CSV File Name	File Content
delivery_logs.csv	Event-level shipment tracking data capturing status updates, delivery attempts, and operational details for each shipment.
ground_truth.csv	Labeled reference dataset providing expected outcomes, escalation decisions, and reasoning aligned with delivery events for evaluation purposes.



Project Evaluation Rubric

Section	Points
Data Loading and Model Initialization	5
Defining Tools	9
Defining the Multi-Agent System Architecture	17
Test Cases Execution, Evaluation, and Observability	17
Conclusions and Business Recommendations	4
Presentation/Notebook - Overall Quality	8

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Load the SQLite database

Load the CSV files

Ingest PDF for RAG

Guiding Questions and Considerations

How do we verify that the database has been loaded correctly (e.g., listing tables, running a simple SELECT query)?

How do we verify that each CSV has been loaded properly (e.g., checking shape, column names, a few sample rows)?

How do we decide the appropriate chunk size and overlap for the document at hand?

This file is meant for personal use by ali@zarreh.ai only.

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Set up the LLM and LangSmith
Traceability

Guiding Questions and Considerations

How do we choose the appropriate OpenAI model (e.g., gpt-4o, gpt-4o-mini) based on the task at hand and credits constraints?

How do we securely manage and load the OpenAI API key?

How do we enable traceability to monitor agent behavior?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Define a list of tools to be used
by the AI Agent

Guiding Questions and Considerations

How do we identify which tasks requires a deterministic vs a probabilistic (LLM-based) execution?

How do we choose a minimal yet sufficient set of tools that cover the required capabilities without making the agent overly complex?

How do we clearly define each tool's inputs, outputs, and purpose so the agent can select and use them correctly?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Tool to:

- read delivery logs
- read customer data from database
- read locker data from database
- retrieve playbook excerpts via RAG

Guiding Questions and Considerations

How do we clearly define the inputs and outputs for each tool so the AI agent can call and interpret them reliably?

How do we structure tool outputs in a uniform format so the AI agent can easily combine information from multiple tools?

How do we uniquely identify the entity (e.g., shipping_id) to avoid any mismatches?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Define the multi-agent architecture

Define Multi-Agent State

Define Sub-agents

Guiding Questions and Considerations

How do we decide which parts of the workflow truly benefit from being split across multiple agents?

How do we design a shared or coordinated multi-agent state (global + per-agent state) so that information flows correctly between agents without duplication, inconsistency, or unnecessary exposure of sensitive data?

What are the key sub-agents, and what role does each play?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Define Prompts

Define Agent Nodes

Define Routing

Guiding Questions and Considerations

How do we craft prompts for each sub-agent that clearly state its role, input format, tools it can use, and expected output format so that agent hand-offs are reliable?

How do we represent agent nodes so that we can trace and debug the flow of control through the system?

How do we define routing logic that decides which agent/node should handle a given request or intermediate result, and under what conditions?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Define Escalation logic

Address data and agent security

Guiding Questions and Considerations

How do we define clear escalation criteria (e.g., low confidence, conflicting outputs, repeated failures, safety flags) that determine when a case should be handed to another agent or a human?

How do we enforce data security and privacy across all agents and tools (e.g., least-privilege access, redaction, and audit logging of sensitive actions)?

How do we prevent agents from accessing or exposing data they are not authorized to see (e.g., role-based permissions, scoped tool access, context filtering)?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Guiding Questions and Considerations

Visualize the workflow

How is the end-to-end workflow represented for clarity and debugging?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Execute all test cases

Print the evaluation metrics for
each test case

Guiding Questions and Considerations

Did the agent successfully execute all test cases end-to-end?

Are evaluation metrics (e.g., accuracy, latency, tool usage) captured for
each test case?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Print traces & document citations
for all test cases

Guiding Questions and Considerations

How do we capture and print traces (agent messages, tool calls, intermediate reasoning) and document citations (sources, URLs, document IDs) in a clear, structured format for every test case?

How do we ensure that traces and citations respect privacy, security, and attribution requirements while still being detailed enough for debugging and evaluation?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation/Notebook - Overall Quality

What to Cover?

Provide observations based on the outputs received for each test case

Compute aggregate metrics combining all test cases

Guiding Questions and Considerations

How do we analyze the outputs for each test case to provide qualitative observations (e.g., patterns of errors, hallucinations, missing citations, good behaviors) alongside quantitative metrics?

How do we relate both per-test and aggregate results back to the initially defined objectives?

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation /
Notebook Quality

What to Cover?

Conclude with the key takeaways
for the business

Guiding Questions and Considerations

What are the key things that we learned from the data that's useful for
the business?

Logical next steps are a key inclusion

Project Evaluation Rubric

Data Loading and
Model Initialization

Defining Tools

Defining the
Multi-Agent System
Architecture

Test Cases
Execution

Conclusions and
Business
Recommendations

Presentation /
Notebook Quality

Low-Code Considerations

Clear structure, flow, and visual appeal - everything sits well in a story

Crispness - Focus on key points

All sections of the rubric included, even if in an appendix

Full-Code Considerations

Clear structure and flow

Well-commented and executed code

All sections of the rubric included, even if in an appendix

Project Submission Guidelines: Low-Code

For learners who aspire to be in managerial roles in the future, focusing on solution review, interpretation, recommendations, and communication with business stakeholders

Download the dataset and the *Learner Notebook - Low Code* (this is a template notebook)

Fill in the blanks in the notebook to complete and execute the code to solve the questions and perform all the tasks as per the grading rubric

Once the notebook is completely executed and necessary outputs obtained, a business presentation (using Microsoft PowerPoint, Google Slides, etc.) has to be created

Project Submission Guidelines: Low-Code

The presentation should contain observations, insights, and recommendations for the business problem

The presentation template provided can be referred to as a sample

Once the presentation is complete, convert the presentation to .pdf format

The presentation should be submitted as a PDF file (.pdf) and NOT as a .pptx file

Please make sure that all the sections mentioned in the grading rubric have been covered in the submission

Project Submission Guidelines: Full-Code

For learners who aspire to be in hands-on coding roles in the future, focusing on building solution from scratch

Download the dataset and the *Learner Notebook - Full Code* (this is a template notebook containing high-level steps to perform and insight-based questions)

Write necessary code to solve the questions and perform all the tasks as per the grading rubric

Please feel free to

- leverage the pre-filled code blocks as they are, or
- update the pre-filled code blocks to incorporate necessary changes as per your desired solution workflow for the business problem at hand, or
- discard the pre-filled code blocks and write the entire code from scratch

Project Submission Guidelines: Full-Code

Clearly write down observations, insights, and recommendations for the business problem based on the analysis

Once the notebook is complete, download it as a **.ipynb** file and convert it to a **.html** file

The notebook should be submitted as an HTML file (.html) and NOT as a notebook file (.ipynb)

The conversion can be done via [free online tools](#)

Please make sure that all the sections mentioned in the grading rubric have been covered in the submission



How do I structure prompts for better defining the tasks for each of the subagents?

Effective prompt structure typically includes:

- Role definition
- Clear task instruction
- Data context (structured)
- Safety boundaries
- Tone requirement

It's important to pre-format inputs, enforce strict output schemas, and explicitly label fields to ensure consistent and reliable agent behavior.

Why am I getting errors while parsing or loading data?

Common issues include:

- Inconsistent column names
- Mixed data types (e.g., strings in numeric fields)
- Date formatting inconsistencies
- Improper key relationships between tables

It's important to validate schemas early, and standardize formats before using data in downstream logic.



Power Ahead!

